Experiment 1: Vector and Matrix Operations for Deep Learning

Question 1

Question:

A hospital uses a deep learning model to classify medical images. A small grayscale image is represented by a 3 × 3 matrix of pixel intensities. Before classification, the image must be converted into a one-dimensional vector, and the total pixel intensity must be calculated. Write a Python program to flatten the image matrix into a one-dimensional vector and calculate the sum of all pixel intensities.

Image matrix: A = [[10, 20, 30], [40, 50, 60], [70, 80, 90]]

Aim:

To flatten a 3 × 3 grayscale image matrix into a one-dimensional vector and to calculate the sum of all pixel intensities using NumPy.

Procedure:

1. Import the NumPy library.
2. Create the 3 × 3 image matrix A as a two-dimensional NumPy array.
3. Convert the matrix into a one-dimensional vector using the flatten() method.
4. Calculate the total pixel intensity using np.sum().
5. Display the original matrix, the flattened vector and the sum of the pixel intensities.

Program:

```
import numpy as np

# Step 1: Image matrix (3 x 3 grayscale image)
A = np.array([[10, 20, 30],
              [40, 50, 60],
              [70, 80, 90]])

# Step 2: Flatten the matrix into a 1-D vector
flat_vector = A.flatten()

# Step 3: Sum of all pixel intensities
total_intensity = np.sum(flat_vector)

# Step 4: Display the results
print("Image matrix A:")
print(A)
print("Flattened vector :", flat_vector)
print("Sum of all pixel intensities :", total_intensity)
```

Output:

```
Image matrix A:
[[10 20 30]
 [40 50 60]
 [70 80 90]]
Flattened vector : [10 20 30 40 50 60 70 80 90]
Sum of all pixel intensities : 450
```

Result:

The program was executed successfully and the output was obtained as shown above.

Question 2

Question:

A deep learning model predicts student performance based on study hours, practice-test scores, and assignment scores. The input features are X = [4, 6, 8], the corresponding weights are W = [0.5, 0.3, 0.2], and the bias is b = 2. Write a Python program to calculate the predicted output by computing the dot product of the input and weight vectors and adding the bias.

Aim:

To calculate the predicted output of a simple neuron, $y = X \cdot W + b$, by computing the dot product of the input and weight vectors and adding the bias.

Procedure:

1. Import the NumPy library.
2. Define the input vector X, the weight vector W and the bias b.
3. Compute the dot product of X and W using np.dot().
4. Add the bias b to the dot product to obtain the predicted output.
5. Display the dot product and the final predicted output.

Program:

```
import numpy as np

# Step 1: Inputs, weights and bias
X = np.array([4, 6, 8])        # study hours, practice-test score, assignment score
W = np.array([0.5, 0.3, 0.2])  # weights
b = 2                          # bias

# Step 2: Dot product of X and W
dot_product = np.dot(X, W)

# Step 3: Add the bias
y_pred = dot_product + b

# Step 4: Display the results
print("Input vector X   :", X)
print("Weight vector W  :", W)
print("Bias b           :", b)
print("Dot product X.W  :", dot_product)
print("Predicted output :", y_pred)
```

Output:

```
Input vector X   : [4 6 8]
Weight vector W  : [0.5 0.3 0.2]
Bias b           : 2
Dot product X.W  : 5.4
Predicted output : 7.4
```

Result:

The program was executed successfully and the output was obtained as shown above.

Question 3

Question:

A weather prediction system receives temperature and humidity measurements from two locations. The input matrix is X = [[30, 60], [35, 70]]. The model uses the weight matrix W = [[0.2, 0.5], [0.4, 0.3]]. Write a Python program to multiply the input matrix by the weight matrix and display the predicted output matrix for both locations.

Aim:

To multiply the input matrix X by the weight matrix W and display the predicted output matrix for both locations.

Procedure:

1. Import the NumPy library.
2. Create the input matrix X and the weight matrix W as 2 × 2 NumPy arrays.
3. Multiply the matrices using np.matmul() (or the @ operator).
4. Store the result as the predicted output matrix.
5. Display X, W and the predicted output for Location 1 and Location 2.

Program:

```
import numpy as np

# Step 1: Input matrix (rows = locations, columns = temperature, humidity)
X = np.array([[30, 60],
              [35, 70]])

# Step 2: Weight matrix
W = np.array([[0.2, 0.5],
              [0.4, 0.3]])

# Step 3: Matrix multiplication
output = np.matmul(X, W)       # same as X @ W

# Step 4: Display the results
print("Input matrix X:")
print(X)
print("\nWeight matrix W:")
print(W)
print("\nPredicted output matrix (X x W):")
print(output)
print("\nLocation 1 output :", output[0])
print("Location 2 output :", output[1])
```

Output:

```
Input matrix X:
[[30 60]
 [35 70]]

Weight matrix W:
[[0.2 0.5]
 [0.4 0.3]]

Predicted output matrix (X x W):
[[30.  33. ]
```

```
 [35.  38.5]]

Location 1 output : [30. 33.]
Location 2 output : [35.  38.5]
```

Result:

The program was executed successfully and the output was obtained as shown above.

Question 4

Question:

A handwriting recognition model adjusts its weights during training to reduce prediction errors. The current weight vector is W = [0.7, 0.5, 0.9], the gradient vector is G = [0.2, 0.1, 0.3], and the learning rate is 0.1. Write a Python program to update the weights using the gradient descent rule and display the updated weight vector.

Aim:

To update the weight vector using the gradient descent rule, W_new = W − η · G, and to display the updated weights.

Procedure:

1. Import the NumPy library.
2. Define the weight vector W, the gradient vector G and the learning rate η (eta).
3. Apply the gradient descent update rule: W_new = W - eta * G.
4. Display the old weights, the gradient and the updated weights.

Program:

```
import numpy as np

# Step 1: Current weights, gradient and learning rate
W = np.array([0.7, 0.5, 0.9])
G = np.array([0.2, 0.1, 0.3])
eta = 0.1

# Step 2: Gradient descent update rule  W_new = W - eta * G
W_new = W - eta * G

# Step 3: Display the results
print("Old weight vector     :", W)
print("Gradient vector       :", G)
print("Learning rate         :", eta)
print("Updated weight vector :", W_new)
```

Output:

```
Old weight vector     : [0.7 0.5 0.9]
Gradient vector       : [0.2 0.1 0.3]
Learning rate         : 0.1
Updated weight vector : [0.68 0.49 0.87]
```

Result:

The program was executed successfully and the output was obtained as shown above.

Question 5

Question:

A recommendation system represents two movies using numerical feature vectors. The first movie is represented by A = [2, 4, 6], and the second movie is represented by B = [1, 3, 5]. The system combines the feature vectors to create a combined representation and calculates the similarity score using their dot product. Write a Python program to perform vector addition and calculate the dot product of the two movie feature vectors. Display both results.

Aim:

To perform vector addition on two movie feature vectors and to calculate their dot product as a similarity score.

Procedure:

1. Import the NumPy library.
2. Define the feature vectors A and B of the two movies.
3. Add the two vectors element-wise (A + B) to get the combined representation.
4. Calculate the dot product of A and B using np.dot() to get the similarity score.
5. Display the combined vector and the dot product.

Program:

```
import numpy as np

# Step 1: Movie feature vectors
A = np.array([2, 4, 6])
B = np.array([1, 3, 5])

# Step 2: Vector addition (combined representation)
combined = A + B

# Step 3: Dot product (similarity score)
similarity = np.dot(A, B)

# Step 4: Display the results
print("Movie A features :", A)
print("Movie B features :", B)
print("Vector addition (A + B) :", combined)
print("Dot product (similarity score) :", similarity)
```

Output:

```
Movie A features : [2 4 6]
Movie B features : [1 3 5]
Vector addition (A + B) : [ 3  7 11]
Dot product (similarity score) : 44
```

Result:

The program was executed successfully and the output was obtained as shown above.

Experiment 2: Covariance, Eigenvalues and Eigenvectors Analysis

Question 1

Question:

A college analyzes the relationship between students' study hours and examination marks to understand how study time is associated with academic performance. The study hours are [2, 4, 6, 8, 10], and the corresponding marks are [35, 45, 60, 75, 85]. Write a Python program to calculate the sample covariance between study hours and marks, and to draw a scatter plot showing their relationship.

Aim:

To calculate the sample covariance between study hours and examination marks and to draw a scatter plot showing their relationship.

Procedure:

1. Import NumPy and Matplotlib.
2. Store the study hours and marks as NumPy arrays.
3. Calculate the sample covariance manually using the formula $\Sigma(x - \bar{x})(y - \bar{y}) / (n - 1)$.
4. Verify the result using np.cov() with ddof = 1.
5. Draw a scatter plot of study hours (x-axis) against marks (y-axis) with labels, title and grid.
6. Interpret the sign of the covariance (positive = marks increase with study hours).

Program:

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Data
hours = np.array([2, 4, 6, 8, 10])
marks = np.array([35, 45, 60, 75, 85])
n = len(hours)

# Step 2: Sample covariance (manual) = sum((x - mean_x)(y - mean_y)) / (n - 1)
cov_manual = np.sum((hours - hours.mean()) * (marks - marks.mean())) / (n - 1)

# Step 3: Sample covariance using NumPy (ddof=1 -> sample covariance)
cov_numpy = np.cov(hours, marks, ddof=1)[0, 1]

print("Mean of study hours       :", hours.mean())
print("Mean of marks             :", marks.mean())
print("Sample covariance (manual):", cov_manual)
print("Sample covariance (NumPy) :", cov_numpy)

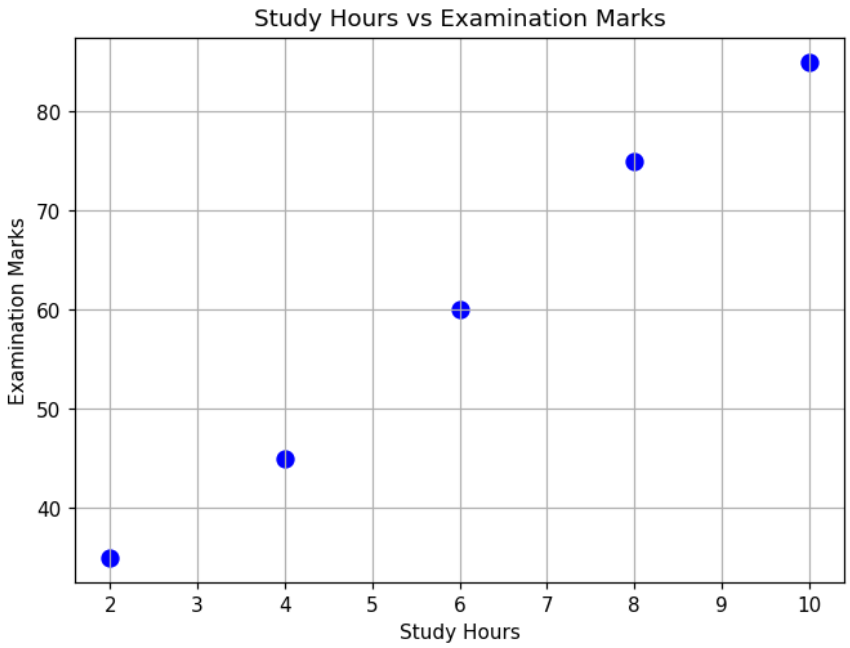
if cov_numpy > 0:
    print("Covariance is positive: marks increase as study hours increase.")

# Step 4: Scatter plot
plt.figure(figsize=(7, 5))
plt.scatter(hours, marks, color="blue", s=70)
plt.xlabel("Study Hours")
plt.ylabel("Examination Marks")
plt.title("Study Hours vs Examination Marks")
plt.grid(True)
```

```
plt.show()
```

Output:

```
Mean of study hours       : 6.0
Mean of marks             : 60.0
Sample covariance (manual): 65.0
Sample covariance (NumPy) : 65.0
Covariance is positive: marks increase as study hours increase.
```



Result:

The program was executed successfully and the output was obtained as shown above.

Question 2

Question:

A weather monitoring system records temperature and humidity over five days. The temperature values are [22, 25, 28, 30, 32], and the humidity values are [80, 75, 68, 60, 55]. Write a Python program to calculate the covariance matrix of temperature and humidity, and to visualize their relationship with a scatter plot.

Aim:

To calculate the covariance matrix of temperature and humidity and to visualize their relationship using a scatter plot.

Procedure:

1. Import NumPy and Matplotlib.
2. Store the temperature and humidity values as NumPy arrays.
3. Compute the 2 × 2 covariance matrix using np.cov().
4. Display the matrix; the diagonal gives the variances and the off-diagonal gives the covariance.
5. Draw a scatter plot of temperature against humidity.
6. Interpret the sign of the covariance (negative = humidity falls as temperature rises).

Program:

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Data
temperature = np.array([22, 25, 28, 30, 32])
humidity = np.array([80, 75, 68, 60, 55])

# Step 2: Covariance matrix
cov_matrix = np.cov(temperature, humidity)

print("Covariance matrix:")
print(cov_matrix)
print("\nVariance of temperature :", cov_matrix[0, 0])
print("Variance of humidity    :", cov_matrix[1, 1])
print("Covariance (T, H)       :", cov_matrix[0, 1])

if cov_matrix[0, 1] < 0:
    print("Covariance is negative: humidity decreases as temperature increases.")

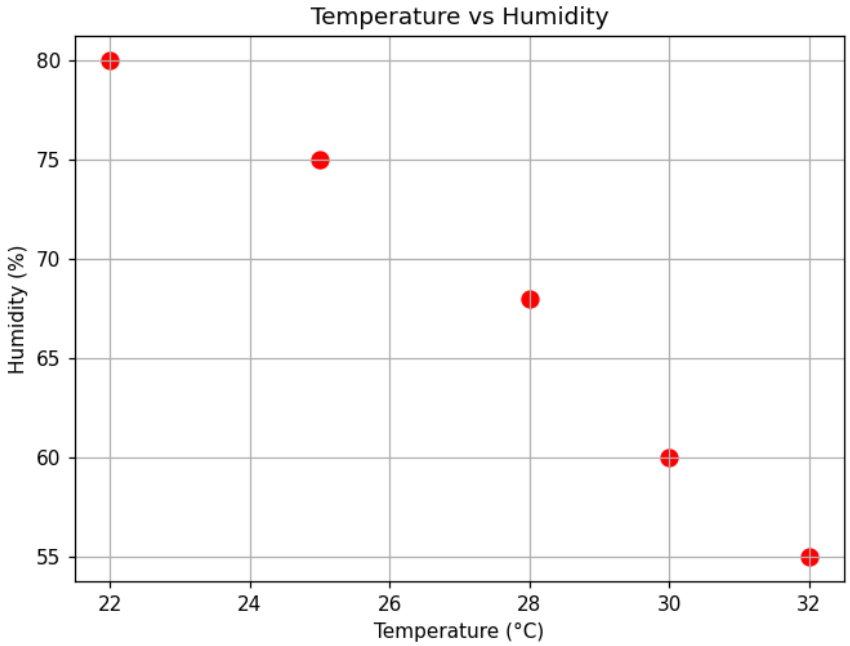
# Step 3: Scatter plot
plt.figure(figsize=(7, 5))
plt.scatter(temperature, humidity, color="red", s=70)
plt.xlabel("Temperature (°C)")
plt.ylabel("Humidity (%)")
plt.title("Temperature vs Humidity")
plt.grid(True)
plt.show()
```

Output:

```
Covariance matrix:
[[ 15.8  -40.55]
```

```
 [-40.55 106.3 ]]

Variance of temperature : 15.799999999999999
Variance of humidity    : 106.3
Covariance (T, H)       : -40.55
Covariance is negative: humidity decreases as temperature increases.
```



Result:

The program was executed successfully and the output was obtained as shown above.

Question 3

Question:

An image-processing system uses the matrix A = [[4, 2], [2, 4]] to analyze patterns in image features. Write a Python program to calculate the eigenvalues of the matrix and to display the eigenvalues in a bar chart.

Aim:

To calculate the eigenvalues of the matrix A and to display them in a bar chart.

Procedure:

1. Import NumPy and Matplotlib.
2. Define the matrix A as a NumPy array.
3. Calculate the eigenvalues using np.linalg.eigvals().
4. Display the eigenvalues.
5. Plot the eigenvalues in a bar chart with labels λ1 and λ2, axis titles and values above the bars.

Program:

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Matrix
A = np.array([[4, 2],
              [2, 4]])

# Step 2: Eigenvalues
eigenvalues = np.linalg.eigvals(A)
eigenvalues = np.sort(eigenvalues)[::-1]      # largest first

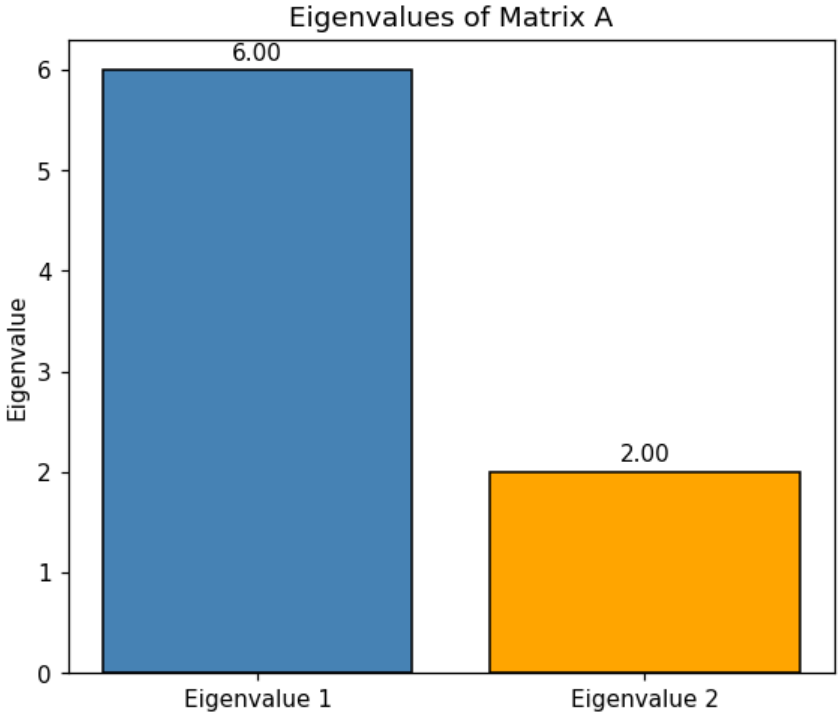
print("Matrix A:")
print(A)
print("\nEigenvalues:", eigenvalues)

# Step 3: Bar chart
labels = ["Eigenvalue 1", "Eigenvalue 2"]
plt.figure(figsize=(6, 5))
bars = plt.bar(labels, eigenvalues, color=["steelblue", "orange"], edgecolor="black")
for bar, val in zip(bars, eigenvalues):
    plt.text(bar.get_x() + bar.get_width() / 2, val + 0.1, f"{val:.2f}", ha="center")
plt.ylabel("Eigenvalue")
plt.title("Eigenvalues of Matrix A")
plt.show()
```

Output:

```
Matrix A:
[[4 2]
 [2 4]]

Eigenvalues: [6. 2.]
```



Result:

The program was executed successfully and the output was obtained as shown above.

Question 4

Question:

A machine learning model uses eigenvectors to identify important directions in a two-dimensional dataset. The covariance matrix is C = [[3, 1], [1, 3]]. Write a Python program to calculate the eigenvalues and eigenvectors of the covariance matrix, and to plot the eigenvectors as arrows from the origin in a two-dimensional coordinate system.

Aim:

To calculate the eigenvalues and eigenvectors of the covariance matrix C and to plot the eigenvectors as arrows from the origin in a two-dimensional coordinate system.

Procedure:

1. Import NumPy and Matplotlib.
2. Define the covariance matrix C.
3. Calculate the eigenvalues and eigenvectors using np.linalg.eig() (each column of the result is one eigenvector).
4. Display the eigenvalues and eigenvectors.
5. Plot each eigenvector as an arrow starting at the origin (0, 0) using plt.quiver().
6. Set equal axis scaling, add axes lines, grid, legend and title, then show the plot.

Program:

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Covariance matrix
C = np.array([[3, 1],
              [1, 3]])

# Step 2: Eigenvalues and eigenvectors (columns of 'eigenvectors')
eigenvalues, eigenvectors = np.linalg.eig(C)

print("Covariance matrix C:")
print(C)
print("\nEigenvalues :", eigenvalues)
print("\nEigenvectors (each column is an eigenvector):")
print(eigenvectors)

# Step 3: Plot eigenvectors as arrows from the origin
colors = ["red", "blue"]
plt.figure(figsize=(6, 6))
for i in range(2):
    v = eigenvectors[:, i]
    plt.quiver(0, 0, v[0], v[1], angles="xy", scale_units="xy", scale=1,
               color=colors[i],
               label=f"v{i+1} = {np.round(v, 3)}, eigenvalue = {eigenvalues[i]:.2f}")

plt.xlim(-1.5, 1.5)
plt.ylim(-1.5, 1.5)
plt.axhline(0, color="black", linewidth=0.8)
plt.axvline(0, color="black", linewidth=0.8)
plt.gca().set_aspect("equal")
plt.grid(True)
plt.xlabel("x")
```

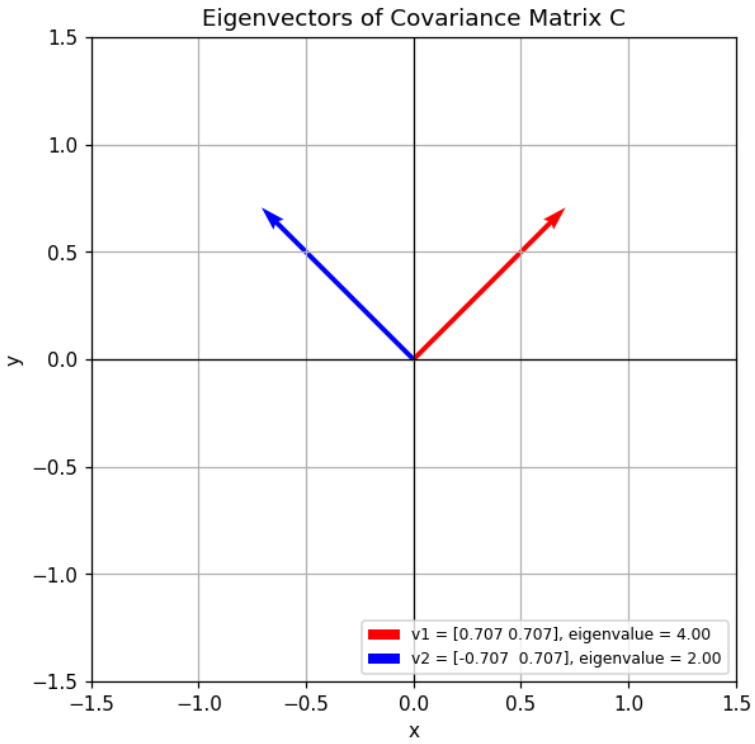
```
plt.ylabel("y")
plt.title("Eigenvectors of Covariance Matrix C")
plt.legend(loc="lower right", fontsize=8)
plt.show()
```

Output:

```
Covariance matrix C:
[[3 1]
 [1 3]]

Eigenvalues : [4. 2.]

Eigenvectors (each column is an eigenvector):
[[ 0.70710678 -0.70710678]
 [ 0.70710678  0.70710678]]
```



Result:

The program was executed successfully and the output was obtained as shown above.

Question 5

Question:

An intelligent monitoring system collects measurements from two sensors. The covariance matrix of the measurements is C = [[5, 2], [2, 2]]. The system uses eigenvalue and eigenvector analysis to identify the principal direction of variation in the sensor data. Write a Python program using NumPy to calculate the eigenvalues and eigenvectors, identify the eigenvector corresponding to the largest eigenvalue, and use Matplotlib to visualize both eigenvectors as arrows on a two-dimensional graph.

Aim:

To calculate the eigenvalues and eigenvectors of the sensor covariance matrix, to identify the principal direction (eigenvector of the largest eigenvalue) and to visualize both eigenvectors as arrows.

Procedure:

1. Import NumPy and Matplotlib.
2. Define the covariance matrix C.
3. Calculate the eigenvalues and eigenvectors using np.linalg.eig().
4. Find the index of the largest eigenvalue using np.argmax().
5. Select the corresponding eigenvector as the principal direction of variation.
6. Plot both eigenvectors as arrows from the origin, highlighting the principal one, and display the graph.

Program:

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Covariance matrix of the two sensors
C = np.array([[5, 2],
              [2, 2]])

# Step 2: Eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(C)

print("Covariance matrix C:")
print(C)
print("\nEigenvalues :", eigenvalues)
print("\nEigenvectors (columns):")
print(eigenvectors)

# Step 3: Principal direction = eigenvector of the largest eigenvalue
idx = np.argmax(eigenvalues)
principal = eigenvectors[:, idx]
print("\nLargest eigenvalue :", eigenvalues[idx])
print("Principal direction (eigenvector) :", np.round(principal, 4))
angle = np.degrees(np.arctan2(principal[1], principal[0]))
print("Angle of principal direction with x-axis :", round(angle, 2), "degrees")

# Step 4: Plot both eigenvectors as arrows
plt.figure(figsize=(6, 6))
for i in range(2):
```

```
    v = eigenvectors[:, i]
    is_principal = (i == idx)
    plt.quiver(0, 0, v[0], v[1], angles="xy", scale_units="xy", scale=1,
               color="red" if is_principal else "blue",
               label=("Principal eigenvector" if is_principal else "Second eigenvector")
                     + f" (eigenvalue = {eigenvalues[i]:.2f})")

plt.xlim(-1.5, 1.5)
plt.ylim(-1.5, 1.5)
plt.axhline(0, color="black", linewidth=0.8)
plt.axvline(0, color="black", linewidth=0.8)
plt.gca().set_aspect("equal")
plt.grid(True)
plt.xlabel("Sensor 1 direction")
plt.ylabel("Sensor 2 direction")
plt.title("Eigenvectors and Principal Direction of Variation")
plt.legend(loc="lower right", fontsize=8)
plt.show()
```

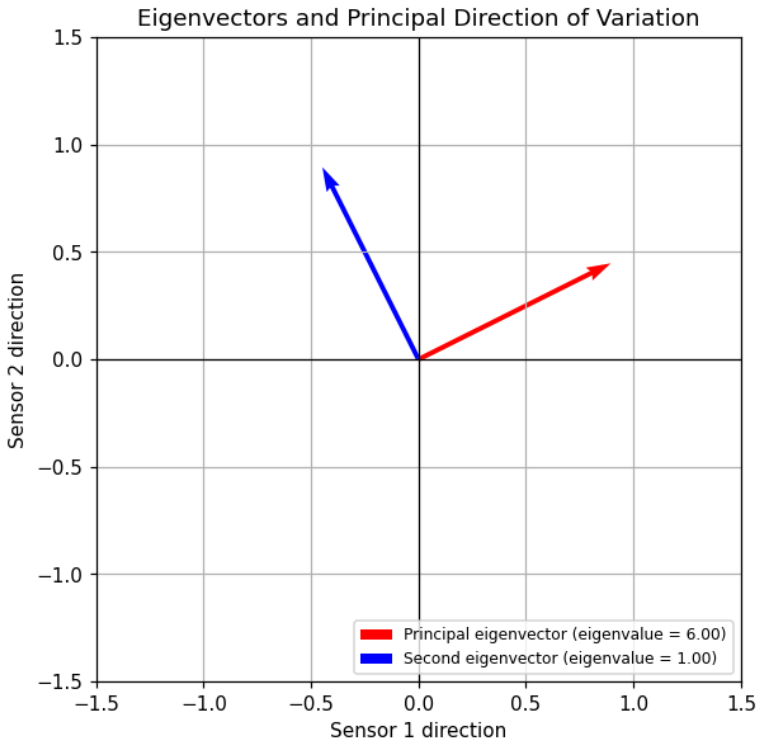
Output:

```
Covariance matrix C:
[[5 2]
 [2 2]]

Eigenvalues : [6. 1.]

Eigenvectors (columns):
[[ 0.89442719 -0.4472136 ]
 [ 0.4472136   0.89442719]]

Largest eigenvalue : 6.0
Principal direction (eigenvector) : [0.8944 0.4472]
Angle of principal direction with x-axis : 26.57 degrees
```



Result:

The program was executed successfully and the output was obtained as shown above.

Experiment 3: Probability Distributions and Bayes' Theorem

Question 1

Question:

A factory produces electronic components, and the probability that a component is defective is 0.1. A quality inspector randomly selects 20 components. The number of defective components follows a binomial distribution. Write a Python program to calculate the probability that exactly 3 components are defective, and to plot the probability mass function for 0 to 20 defective components.

Aim:

To calculate the binomial probability of exactly 3 defective components out of 20 (p = 0.1) and to plot the probability mass function for 0 to 20 defective components.

Procedure:

1. Import the math and Matplotlib libraries.
2. Define n = 20, p = 0.1 and k = 3.
3. Define the binomial PMF: P(X = k) = C(n, k) · p^k · (1 − p)^(n − k).
4. Calculate P(X = 3) and display it.
5. Calculate the PMF for every k from 0 to 20.
6. Plot the PMF as a bar chart, highlighting k = 3.

Program:

```
import math
import matplotlib.pyplot as plt

# Step 1: Parameters
n = 20      # number of components inspected
p = 0.1     # probability that a component is defective

# Step 2: Binomial PMF  P(X = k) = C(n, k) * p^k * (1 - p)^(n - k)
def binomial_pmf(k, n, p):
    return math.comb(n, k) * (p ** k) * ((1 - p) ** (n - k))

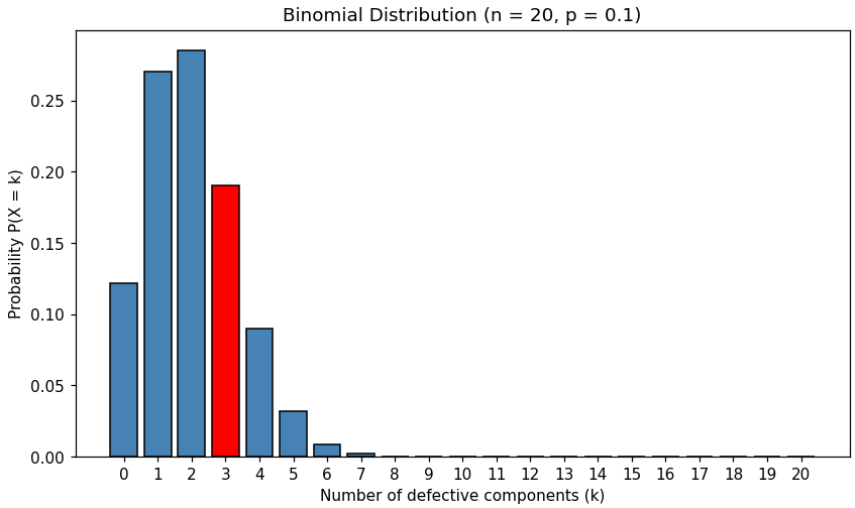
# Step 3: Probability of exactly 3 defective components
prob_3 = binomial_pmf(3, n, p)
print("P(exactly 3 defective) =", round(prob_3, 4))

# Step 4: PMF for 0 to 20 defective components
k_values = list(range(0, n + 1))
pmf_values = [binomial_pmf(k, n, p) for k in k_values]
print("Sum of all probabilities =", round(sum(pmf_values), 4))

# Step 5: Plot the PMF
colors = ["red" if k == 3 else "steelblue" for k in k_values]
plt.figure(figsize=(9, 5))
plt.bar(k_values, pmf_values, color=colors, edgecolor="black")
plt.xlabel("Number of defective components (k)")
plt.ylabel("Probability P(X = k)")
plt.title("Binomial Distribution (n = 20, p = 0.1)")
plt.xticks(k_values)
plt.show()
```

Output:

```
P(exactly 3 defective) = 0.1901
Sum of all probabilities = 1.0
```



Result:

The program was executed successfully and the output was obtained as shown above.

Question 2

Question:

A lecturer observes that students' examination marks approximately follow a normal distribution with a mean of 70 and a standard deviation of 10. Write a Python program to calculate the probability that a randomly selected student scores between 60 and 80, and to plot the normal distribution with the interval highlighted.

Aim:

To calculate the probability that a student scores between 60 and 80 when marks follow a normal distribution ($\mu = 70$, $\sigma = 10$) and to plot the distribution with this interval highlighted.

Procedure:

1. Import NumPy, math and Matplotlib.
2. Define the mean $\mu = 70$, the standard deviation $\sigma = 10$ and the interval limits 60 and 80.
3. Define the normal CDF using the error function: $F(x) = 0.5 \cdot [1 + \text{erf}((x - \mu) / (\sigma\sqrt{2}))]$.
4. Calculate $P(60 < X < 80) = F(80) - F(60)$ and display it.
5. Compute the normal probability density curve over the range $\mu \pm 4\sigma$.
6. Plot the curve and shade the area between 60 and 80.

Program:

```
import math
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Parameters
mu = 70
sigma = 10
a, b = 60, 80

# Step 2: Normal CDF using the error function
def normal_cdf(x, mu, sigma):
    return 0.5 * (1 + math.erf((x - mu) / (sigma * math.sqrt(2))))

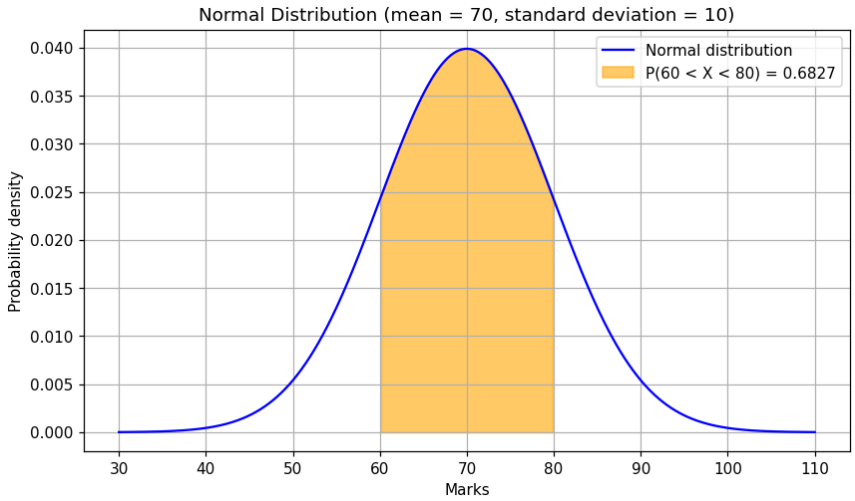
# Step 3: P(60 < X < 80) = F(80) - F(60)
prob = normal_cdf(b, mu, sigma) - normal_cdf(a, mu, sigma)
print("P(60 < X < 80) =", round(prob, 4))
print("Percentage     =", round(prob * 100, 2), "%")

# Step 4: Normal probability density curve
x = np.linspace(mu - 4 * sigma, mu + 4 * sigma, 500)
pdf = (1 / (sigma * np.sqrt(2 * np.pi))) * np.exp(-0.5 * ((x - mu) / sigma) ** 2)

# Step 5: Plot with the interval highlighted
plt.figure(figsize=(9, 5))
plt.plot(x, pdf, color="blue", label="Normal distribution")
plt.fill_between(x, pdf, where=(x >= a) & (x <= b), color="orange", alpha=0.6,
                 label=f"P({a} < X < {b}) = {prob:.4f}")
plt.xlabel("Marks")
plt.ylabel("Probability density")
plt.title("Normal Distribution (mean = 70, standard deviation = 10)")
plt.legend()
plt.grid(True)
plt.show()
```

Output:

```
P(60 < X < 80) = 0.6827
Percentage     = 68.27 %
```



Result:

The program was executed successfully and the output was obtained as shown above.

Question 3

Question:

A service centre receives an average of 4 customers every 10 minutes. The number of customer arrivals follows a Poisson distribution. Write a Python program to calculate the probability that exactly 6 customers arrive in a 10-minute interval, and to plot the probability mass function for customer arrivals from 0 to 12.

Aim:

To calculate the Poisson probability that exactly 6 customers arrive in a 10-minute interval ($\lambda = 4$) and to plot the probability mass function for 0 to 12 arrivals.

Procedure:

1. Import the math and Matplotlib libraries.
2. Define the average arrival rate $\lambda = 4$ and $k = 6$.
3. Define the Poisson PMF: $P(X = k) = e^{(-\lambda)} \cdot \lambda^k / k!$.
4. Calculate $P(X = 6)$ and display it.
5. Calculate the PMF for $k = 0$ to 12.
6. Plot the PMF as a bar chart, highlighting $k = 6$.

Program:

```
import math
import matplotlib.pyplot as plt

# Step 1: Parameters
lam = 4     # average number of arrivals in 10 minutes

# Step 2: Poisson PMF  P(X = k) = e^(-lambda) * lambda^k / k!
def poisson_pmf(k, lam):
    return math.exp(-lam) * (lam ** k) / math.factorial(k)

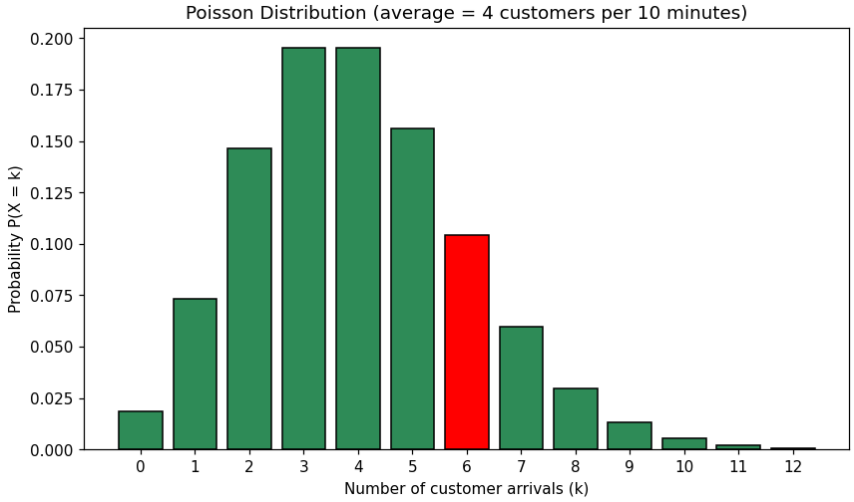
# Step 3: Probability of exactly 6 arrivals
prob_6 = poisson_pmf(6, lam)
print("P(exactly 6 customers) =", round(prob_6, 4))

# Step 4: PMF for 0 to 12 arrivals
k_values = list(range(0, 13))
pmf_values = [poisson_pmf(k, lam) for k in k_values]

# Step 5: Plot the PMF
colors = ["red" if k == 6 else "seagreen" for k in k_values]
plt.figure(figsize=(9, 5))
plt.bar(k_values, pmf_values, color=colors, edgecolor="black")
plt.xlabel("Number of customer arrivals (k)")
plt.ylabel("Probability P(X = k)")
plt.title("Poisson Distribution (average = 4 customers per 10 minutes)")
plt.xticks(k_values)
plt.show()
```

Output:

```
P(exactly 6 customers) = 0.1042
```



Result:

The program was executed successfully and the output was obtained as shown above.

Question 4

Question:

A disease affects 2% of a population. A medical test correctly identifies a person with the disease 95% of the time. The test also produces a positive result for 5% of people who do not have the disease. Write a Python program to apply Bayes' theorem and calculate the probability that a person actually has the disease given that the test result is positive. Use Matplotlib to compare the prior probability of having the disease with the posterior probability after a positive test result.

Aim:

To apply Bayes' theorem to find the probability that a person has the disease given a positive test result, and to compare the prior and posterior probabilities using a bar chart.

Procedure:

1. Import Matplotlib.
2. Define P(D) = 0.02, P(+|D) = 0.95 and P(+|not D) = 0.05.
3. Calculate P(not D) = 1 − P(D).
4. Calculate the total probability of a positive test: P(+) = P(+|D)P(D) + P(+|not D)P(not D).
5. Apply Bayes' theorem: P(D|+) = P(+|D)P(D) / P(+).
6. Plot the prior P(D) and posterior P(D|+) in a bar chart and display the result.

Program:

```
import matplotlib.pyplot as plt

# Step 1: Given probabilities
p_disease = 0.02              # P(D)   prior probability
p_pos_given_disease = 0.95    # P(+|D) sensitivity
p_pos_given_healthy = 0.05    # P(+|not D) false positive rate

# Step 2: Probability of no disease
p_healthy = 1 - p_disease

# Step 3: Total probability of a positive test
p_positive = p_pos_given_disease * p_disease + p_pos_given_healthy * p_healthy

# Step 4: Bayes' theorem  P(D|+) = P(+|D) * P(D) / P(+)
posterior = (p_pos_given_disease * p_disease) / p_positive

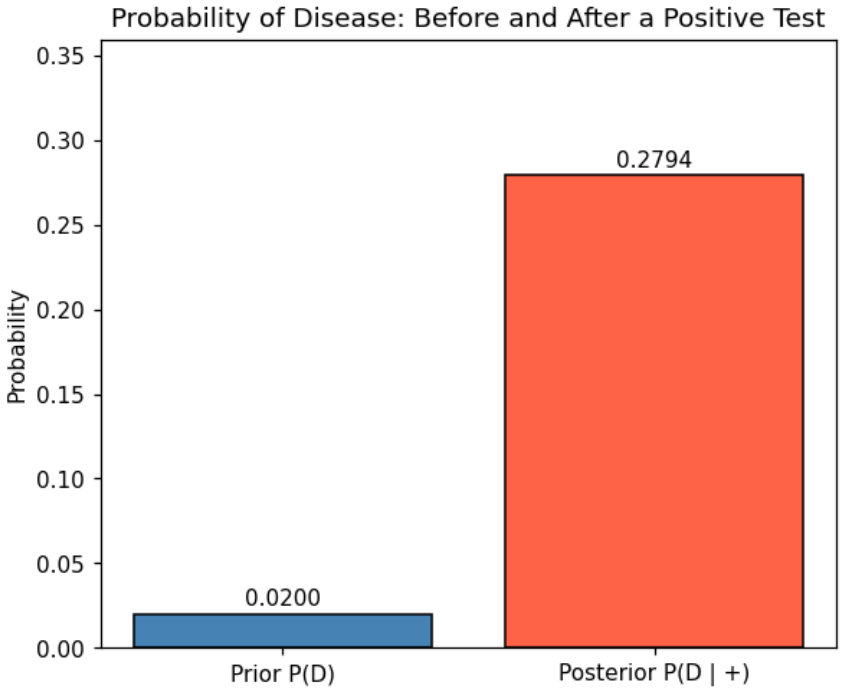
print("Prior probability P(D)            =", p_disease)
print("Total probability of positive P(+) =", round(p_positive, 4))
print("Posterior probability P(D | +)     =", round(posterior, 4))
print("Posterior in percentage            =", round(posterior * 100, 2), "%")

# Step 5: Compare prior and posterior
labels = ["Prior P(D)", "Posterior P(D | +)"]
values = [p_disease, posterior]
plt.figure(figsize=(6, 5))
bars = plt.bar(labels, values, color=["steelblue", "tomato"], edgecolor="black")
for bar, val in zip(bars, values):
    plt.text(bar.get_x() + bar.get_width() / 2, val + 0.005, f"{val:.4f}", ha="center")
plt.ylabel("Probability")
plt.title("Probability of Disease: Before and After a Positive Test")
plt.ylim(0, max(values) + 0.08)
```

```
plt.show()
```

Output:

```
Prior probability P(D)            = 0.02
Total probability of positive P(+) = 0.068
Posterior probability P(D | +)     = 0.2794
Posterior in percentage            = 27.94 %
```



Result:

The program was executed successfully and the output was obtained as shown above.

Question 5

Question:

An email filtering system identifies 30% of incoming emails as spam. The word "prize" appears in 70% of spam emails and in 10% of non-spam emails. Write a Python program to apply Bayes' theorem and calculate the probability that an email is spam given that it contains the word "prize." Use Matplotlib to visualize the prior probability of spam and the posterior probability after observing the word.

Aim:

To apply Bayes' theorem to find the probability that an email is spam given that it contains the word "prize", and to visualize the prior and posterior probabilities.

Procedure:

1. Import Matplotlib.
2. Define P(Spam) = 0.30, P(prize|Spam) = 0.70 and P(prize|Not Spam) = 0.10.
3. Calculate P(Not Spam) = 1 − P(Spam).
4. Calculate the total probability of the word "prize": P(prize) = P(prize|Spam)P(Spam) + P(prize|Not Spam)P(Not Spam).
5. Apply Bayes' theorem: P(Spam|prize) = P(prize|Spam)P(Spam) / P(prize).
6. Plot the prior and posterior probabilities in a bar chart and display the result.

Program:

```
import matplotlib.pyplot as plt

# Step 1: Given probabilities
p_spam = 0.30                    # P(Spam)  prior
p_word_given_spam = 0.70         # P(prize | Spam)
p_word_given_not_spam = 0.10     # P(prize | Not Spam)

# Step 2: Probability of non-spam
p_not_spam = 1 - p_spam

# Step 3: Total probability that an email contains the word "prize"
p_word = p_word_given_spam * p_spam + p_word_given_not_spam * p_not_spam

# Step 4: Bayes' theorem  P(Spam | prize)
posterior = (p_word_given_spam * p_spam) / p_word

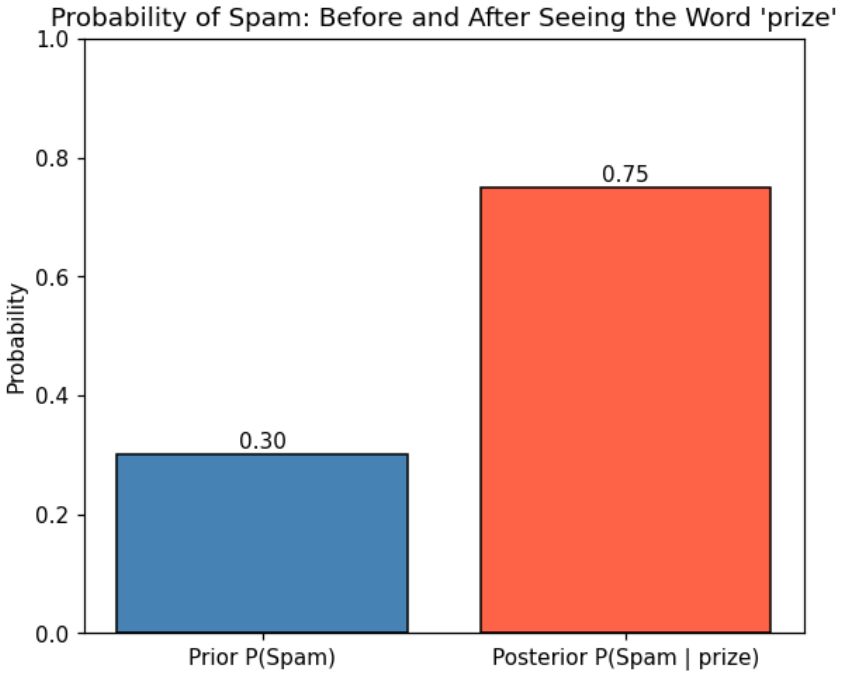
print("Prior probability P(Spam)             =", p_spam)
print("Probability of word 'prize' P(prize)  =", round(p_word, 4))
print("Posterior probability P(Spam | prize) =", round(posterior, 4))
print("Posterior in percentage               =", round(posterior * 100, 2), "%")

# Step 5: Visualize prior vs posterior
labels = ["Prior P(Spam)", "Posterior P(Spam | prize)"]
values = [p_spam, posterior]
plt.figure(figsize=(6, 5))
bars = plt.bar(labels, values, color=["steelblue", "tomato"], edgecolor="black")
for bar, val in zip(bars, values):
    plt.text(bar.get_x() + bar.get_width() / 2, val + 0.01, f"{val:.2f}", ha="center")
plt.ylabel("Probability")
plt.title("Probability of Spam: Before and After Seeing the Word 'prize'")
```

```
plt.ylim(0, 1)
plt.show()
```

Output:

```
Prior probability P(Spam)             = 0.3
Probability of word 'prize' P(prize)  = 0.28
Posterior probability P(Spam | prize) = 0.75
Posterior in percentage               = 75.0 %
```



Result:

The program was executed successfully and the output was obtained as shown above.

Experiment 4: Entropy, Cross-Entropy and KL Divergence

Question 1

Question:

A weather prediction system estimates the probabilities of three possible conditions as sunny, rainy, and cloudy: P = [0.6, 0.3, 0.1]. The system needs to measure the uncertainty in its predictions. Write a Python program to calculate the entropy of the probability distribution and to display the probabilities of the three weather conditions in a bar chart.

Aim:

To calculate the entropy of a weather probability distribution (a measure of uncertainty) and to display the probabilities of the three weather conditions in a bar chart.

Procedure:

1. Import NumPy and Matplotlib.
2. Define the probability distribution P for sunny, rainy and cloudy.
3. Calculate the entropy $H(P) = -\Sigma\, p \cdot \log2(p)$ (in bits).
4. Display the entropy value.
5. Plot the three probabilities in a bar chart with labels and values on the bars.

Program:

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Probability distribution
conditions = ["Sunny", "Rainy", "Cloudy"]
P = np.array([0.6, 0.3, 0.1])

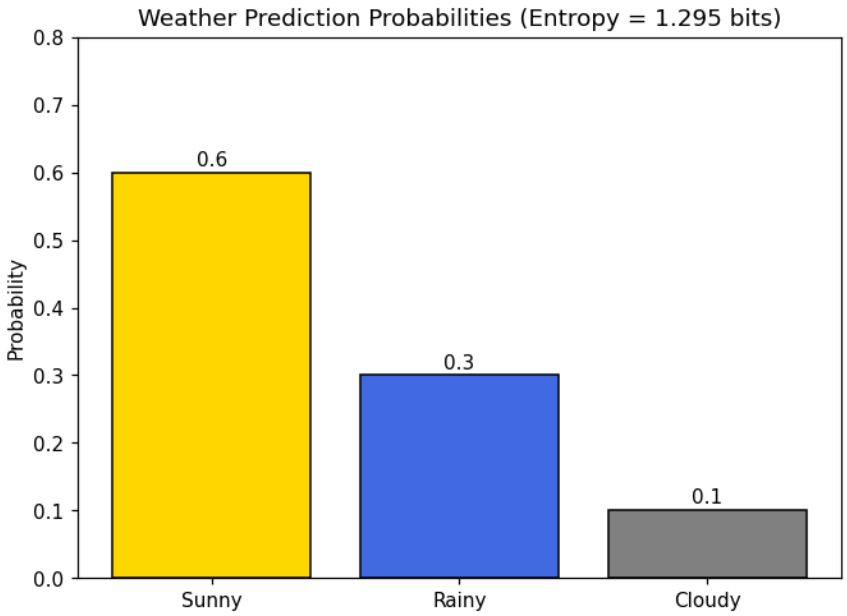
# Step 2: Entropy  H(P) = - sum( p * log2(p) )   (unit: bits)
entropy = -np.sum(P * np.log2(P))

print("Probabilities :", P)
print("Entropy H(P)  =", round(entropy, 4), "bits")

# Step 3: Bar chart of the probabilities
plt.figure(figsize=(7, 5))
bars = plt.bar(conditions, P, color=["gold", "royalblue", "gray"], edgecolor="black")
for bar, val in zip(bars, P):
    plt.text(bar.get_x() + bar.get_width() / 2, val + 0.01, str(val), ha="center")
plt.ylabel("Probability")
plt.title(f"Weather Prediction Probabilities (Entropy = {entropy:.3f} bits)")
plt.ylim(0, 0.8)
plt.show()
```

Output:

```
Probabilities : [0.6 0.3 0.1]
Entropy H(P)  = 1.2955 bits
```



Result:

The program was executed successfully and the output was obtained as shown above.

Question 2

Question:

A deep learning model classifies an image into one of three categories: cat, dog, or bird. The actual class is represented by Y = [1, 0, 0], while the model predicts probabilities P = [0.7, 0.2, 0.1]. Write a Python program to calculate the cross-entropy loss between the actual class distribution and the predicted probabilities, and to compare the actual and predicted probabilities.

Aim:

To calculate the cross-entropy loss between the actual class distribution Y and the predicted probabilities P, and to compare the actual and predicted probabilities using a grouped bar chart.

Procedure:

1. Import NumPy and Matplotlib.
2. Define the actual (one-hot) distribution Y and the predicted distribution P.
3. Calculate the cross-entropy loss $H(Y, P) = -\Sigma\, Y \cdot \ln(P)$ (in nats).
4. Display the loss value.
5. Plot Y and P side by side for the classes cat, dog and bird in a grouped bar chart.

Program:

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Actual and predicted distributions
classes = ["Cat", "Dog", "Bird"]
Y = np.array([1, 0, 0])            # actual class (one-hot)
P = np.array([0.7, 0.2, 0.1])      # predicted probabilities

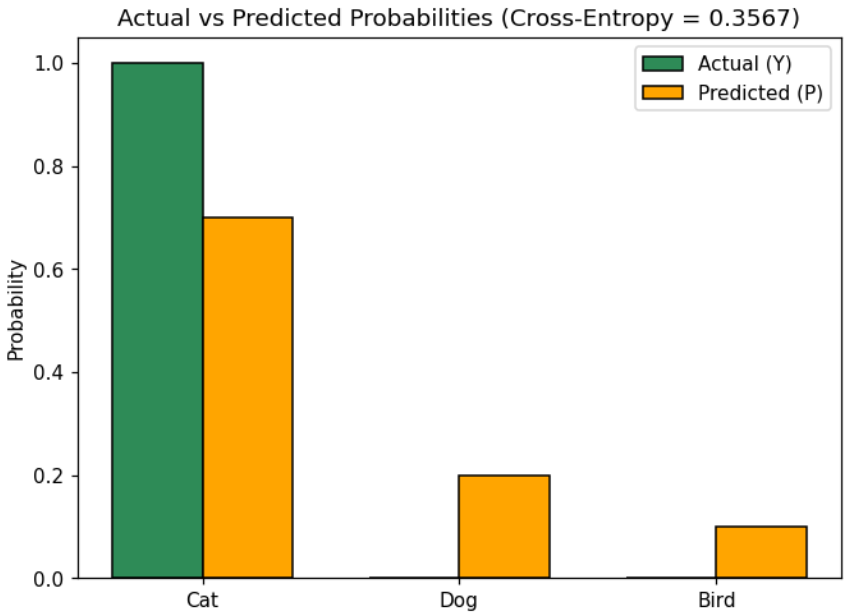
# Step 2: Cross-entropy loss  H(Y, P) = - sum( Y * ln(P) )
cross_entropy = -np.sum(Y * np.log(P))

print("Actual distribution Y    :", Y)
print("Predicted distribution P :", P)
print("Cross-entropy loss       =", round(cross_entropy, 4))

# Step 3: Grouped bar chart comparing actual and predicted
x = np.arange(len(classes))
width = 0.35
plt.figure(figsize=(7, 5))
plt.bar(x - width / 2, Y, width, label="Actual (Y)", color="seagreen", edgecolor="black")
plt.bar(x + width / 2, P, width, label="Predicted (P)", color="orange", edgecolor="black")
plt.xticks(x, classes)
plt.ylabel("Probability")
plt.title(f"Actual vs Predicted Probabilities (Cross-Entropy = {cross_entropy:.4f})")
plt.legend()
plt.show()
```

Output:

```
Actual distribution Y    : [1 0 0]
Predicted distribution P : [0.7 0.2 0.1]
Cross-entropy loss       = 0.3567
```



Result:

The program was executed successfully and the output was obtained as shown above.

Question 3

Question:

Two language models predict the next word using the probability distributions P = [0.5, 0.3, 0.2] and Q = [0.4, 0.4, 0.2]. A developer wants to measure how much the second model's distribution differs from the first. Write a Python program to calculate the KL divergence and to compare the two probability distributions with a grouped bar chart.

Aim:

To calculate the KL divergence between two language-model probability distributions P and Q and to compare them using a grouped bar chart.

Procedure:

1. Import NumPy and Matplotlib.
2. Define the probability distributions P and Q.
3. Calculate the KL divergence $D_{KL}(P \| Q) = \Sigma P \cdot \ln(P / Q)$.
4. Display the KL divergence (also the reverse divergence $D_{KL}(Q \| P)$ to show it is asymmetric).
5. Plot P and Q for the three words in a grouped bar chart.

Program:

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Probability distributions of the two models
words = ["Word 1", "Word 2", "Word 3"]
P = np.array([0.5, 0.3, 0.2])
Q = np.array([0.4, 0.4, 0.2])

# Step 2: KL divergence  D_KL(P || Q) = sum( P * ln(P / Q) )
kl_pq = np.sum(P * np.log(P / Q))
kl_qp = np.sum(Q * np.log(Q / P))      # reverse direction (for comparison)

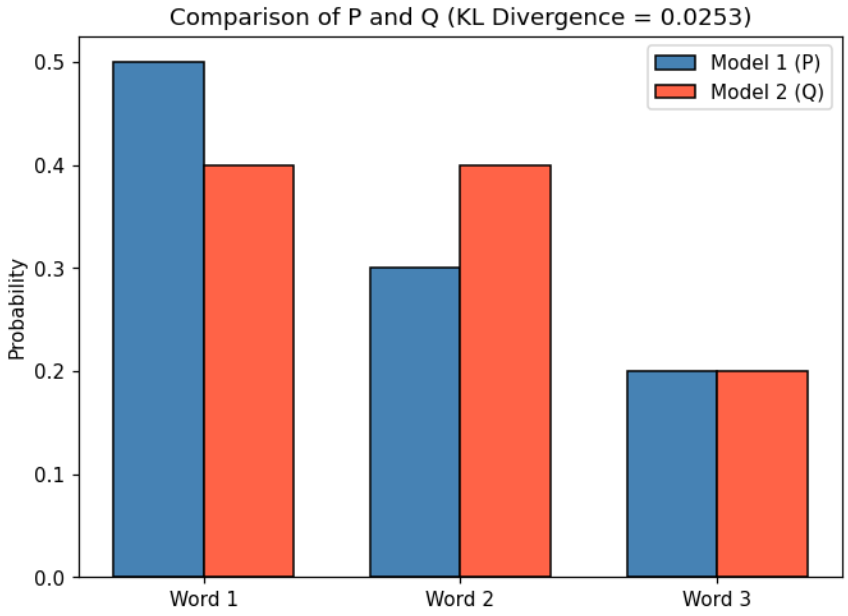
print("P =", P)
print("Q =", Q)
print("KL divergence D_KL(P || Q) =", round(kl_pq, 4))
print("KL divergence D_KL(Q || P) =", round(kl_qp, 4), "(KL divergence is not symmetric)")

# Step 3: Grouped bar chart
x = np.arange(len(words))
width = 0.35
plt.figure(figsize=(7, 5))
plt.bar(x - width / 2, P, width, label="Model 1 (P)", color="steelblue", edgecolor="black")
plt.bar(x + width / 2, Q, width, label="Model 2 (Q)", color="tomato", edgecolor="black")
plt.xticks(x, words)
plt.ylabel("Probability")
plt.title(f"Comparison of P and Q (KL Divergence = {kl_pq:.4f})")
plt.legend()
plt.show()
```

Output:

```
P = [0.5 0.3 0.2]
Q = [0.4 0.4 0.2]
KL divergence D_KL(P || Q) = 0.0253
```

```
KL divergence D_KL(Q || P) = 0.0258 (KL divergence is not symmetric)
```



Result:

The program was executed successfully and the output was obtained as shown above.

Question 4

Question:

A medical decision-support system predicts whether a patient has a disease or not, with probabilities P = [0.9, 0.1]. Another patient's predicted probabilities are Q = [0.5, 0.5]. Write a Python program to calculate the entropy of both predictions and to compare their entropy values in a bar chart. Identify which prediction has greater uncertainty.

Aim:

To calculate the entropy of two disease predictions, to compare their entropy values in a bar chart and to identify which prediction has greater uncertainty.

Procedure:

1. Import NumPy and Matplotlib.
2. Define the prediction distributions P and Q.
3. Calculate the entropy of each: $H = -\Sigma\, p \cdot \log2(p)$ (in bits).
4. Display both entropy values.
5. Compare the values and print which prediction has greater uncertainty.
6. Plot both entropy values in a bar chart.

Program:

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Predicted probabilities [disease, no disease]
P = np.array([0.9, 0.1])     # Patient 1
Q = np.array([0.5, 0.5])     # Patient 2

# Step 2: Entropy function (bits)
def entropy(dist):
    return -np.sum(dist * np.log2(dist))

H_P = entropy(P)
H_Q = entropy(Q)

print("Entropy of Patient 1 prediction P =", round(H_P, 4), "bits")
print("Entropy of Patient 2 prediction Q =", round(H_Q, 4), "bits")

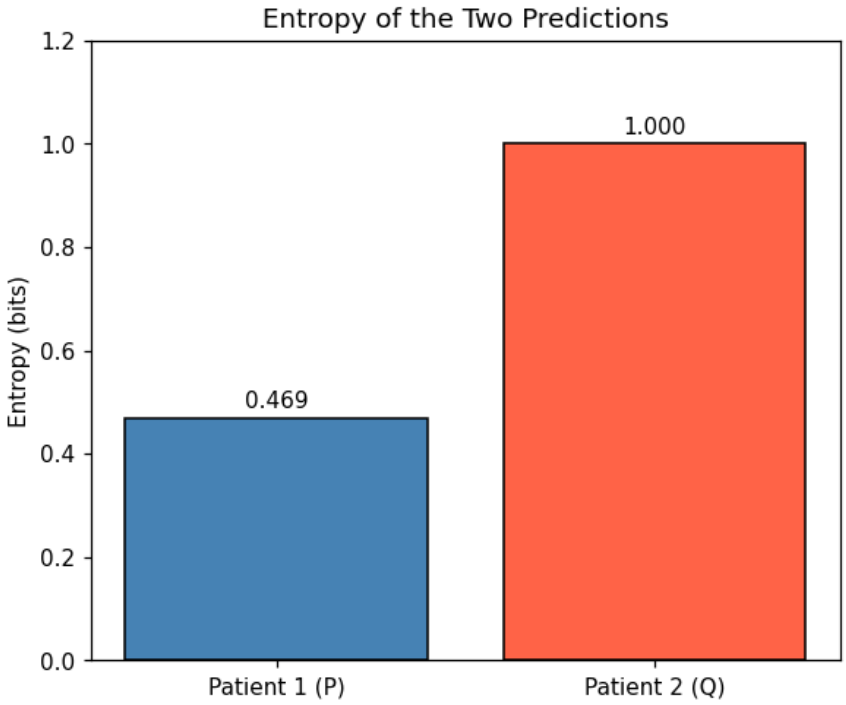
# Step 3: Identify the more uncertain prediction
if H_Q > H_P:
    print("Patient 2 (Q) has greater uncertainty (higher entropy).")
elif H_P > H_Q:
    print("Patient 1 (P) has greater uncertainty (higher entropy).")
else:
    print("Both predictions have equal uncertainty.")

# Step 4: Bar chart
labels = ["Patient 1 (P)", "Patient 2 (Q)"]
values = [H_P, H_Q]
plt.figure(figsize=(6, 5))
bars = plt.bar(labels, values, color=["steelblue", "tomato"], edgecolor="black")
for bar, val in zip(bars, values):
    plt.text(bar.get_x() + bar.get_width() / 2, val + 0.02, f"{val:.3f}", ha="center")
plt.ylabel("Entropy (bits)")
```

```
plt.title("Entropy of the Two Predictions")
plt.ylim(0, 1.2)
plt.show()
```

Output:

```
Entropy of Patient 1 prediction P = 0.469 bits
Entropy of Patient 2 prediction Q = 1.0 bits
Patient 2 (Q) has greater uncertainty (higher entropy).
```



Result:

The program was executed successfully and the output was obtained as shown above.

Question 5

Question:

A neural network classifies an image into four categories. The actual class distribution is Y = [0, 1, 0, 0], and the predicted probability distribution is P = [0.1, 0.6, 0.2, 0.1]. The developer wants to evaluate the prediction error and compare the predicted distribution with the actual distribution. Write a Python program to calculate the cross-entropy loss and KL divergence between the actual and predicted distributions, and to visualize the two distributions.

Aim:

To calculate the cross-entropy loss and the KL divergence between the actual and predicted distributions of a four-class classifier and to visualize the two distributions.

Procedure:

1. Import NumPy and Matplotlib.
2. Define the actual distribution Y and the predicted distribution P.
3. Calculate the cross-entropy loss $H(Y, P) = -\Sigma\, Y \cdot \ln(P)$, using only the terms where $Y > 0$.
4. Calculate the KL divergence $D_{KL}(Y \,||\, P) = \Sigma\, Y \cdot \ln(Y / P)$, again only for $Y > 0$ (since $0 \cdot \ln 0 = 0$).
5. Display both values.
6. Plot the actual and predicted distributions in a grouped bar chart.

Program:

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Actual and predicted distributions
classes = ["Class 1", "Class 2", "Class 3", "Class 4"]
Y = np.array([0, 1, 0, 0])
P = np.array([0.1, 0.6, 0.2, 0.1])

# Step 2: Use only the terms where Y > 0 (because 0 * log(0) is taken as 0)
mask = Y > 0

# Cross-entropy  H(Y, P) = - sum( Y * ln(P) )
cross_entropy = -np.sum(Y[mask] * np.log(P[mask]))

# KL divergence  D_KL(Y || P) = sum( Y * ln(Y / P) )
kl_div = np.sum(Y[mask] * np.log(Y[mask] / P[mask]))

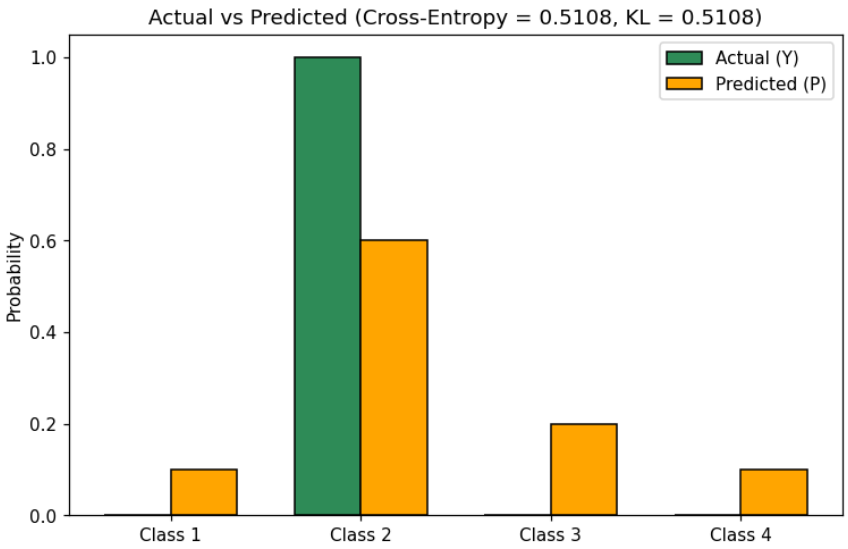
print("Actual distribution Y    :", Y)
print("Predicted distribution P :", P)
print("Cross-entropy loss       =", round(cross_entropy, 4))
print("KL divergence D_KL(Y||P) =", round(kl_div, 4))
print("(For a one-hot Y, the entropy of Y is 0, so cross-entropy = KL divergence.)")

# Step 3: Visualize the two distributions
x = np.arange(len(classes))
width = 0.35
plt.figure(figsize=(8, 5))
plt.bar(x - width / 2, Y, width, label="Actual (Y)", color="seagreen", edgecolor="black")
```

```
plt.bar(x + width / 2, P, width, label="Predicted (P)", color="orange", edgecolor="black")
plt.xticks(x, classes)
plt.ylabel("Probability")
plt.title(f"Actual vs Predicted (Cross-Entropy = {cross_entropy:.4f}, KL = {kl_div:.4f})")
plt.legend()
plt.show()
```

Output:

```
Actual distribution Y    : [0 1 0 0]
Predicted distribution P : [0.1 0.6 0.2 0.1]
Cross-entropy loss       = 0.5108
KL divergence D_KL(Y||P) = 0.5108
(For a one-hot Y, the entropy of Y is 0, so cross-entropy = KL divergence.)
```



Result:

The program was executed successfully and the output was obtained as shown above.

Experiment 5: Bias–Variance, Overfitting and Underfitting Analysis

Question 1

Question:

A real estate company develops machine learning models to predict house prices based on house area. A linear regression model and polynomial regression models of degrees 2 and 10 are trained using the same dataset. The company wants to compare how model complexity affects prediction performance. Write a Python program to generate sample house-price data, train polynomial regression models of degrees 1, 2, and 10, and to plot their prediction curves against the actual data.

Aim:

To generate sample house-price data, to train polynomial regression models of degrees 1, 2 and 10, and to plot their prediction curves against the actual data in order to study the effect of model complexity.

Procedure:

1. Import NumPy and Matplotlib and set a random seed so the data is reproducible.
2. Generate sample data: house area (sq. ft) and price (lakhs) with a non-linear relationship plus random noise.
3. Scale the area values to the range [−1, 1] so that high-degree polynomials are numerically stable.
4. Train polynomial regression models of degrees 1, 2 and 10 using np.polyfit() and calculate the training MSE of each.
5. Generate a smooth line of area values and predict the price with each model using np.polyval().
6. Plot the actual data points and the three prediction curves, and interpret underfitting (degree 1), good fit (degree 2) and overfitting (degree 10).

Program:

```
import warnings
import numpy as np
import matplotlib.pyplot as plt

warnings.simplefilter("ignore")      # hide numerical warnings for high-degree fits
np.random.seed(1)

# Step 1: Generate sample house-price data (area in sq.ft, price in lakhs)
n = 30
area = np.sort(np.random.uniform(500, 2500, n))
price = 30 + 0.05 * area - 0.000012 * area ** 2 + np.random.normal(0, 6, n)

# Step 2: Scale the area to [-1, 1] (keeps high-degree fitting stable)
x = (area - 1500) / 1000

# Step 3: Train polynomial models of degrees 1, 2 and 10 and plot them
degrees = [1, 2, 10]
colors = ["red", "green", "purple"]
```

```
x_line = np.linspace(x.min(), x.max(), 300)

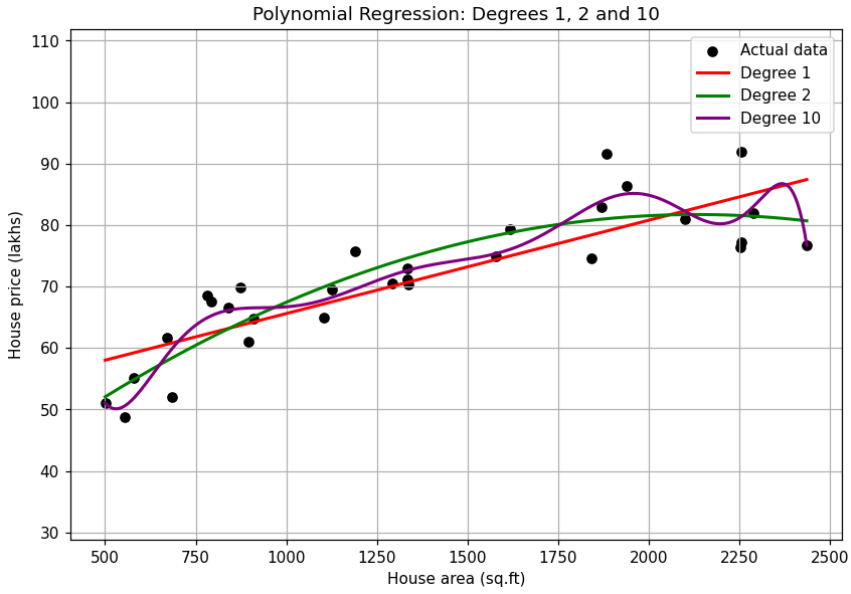
plt.figure(figsize=(9, 6))
plt.scatter(area, price, color="black", label="Actual data")

for d, c in zip(degrees, colors):
    coef = np.polyfit(x, price, d)                        # train the model
    train_mse = np.mean((price - np.polyval(coef, x)) ** 2)
    print(f"Degree {d:2d} -> Training MSE = {train_mse:.2f}")
    plt.plot(x_line * 1000 + 1500, np.polyval(coef, x_line), color=c, linewidth=2,
             label=f"Degree {d}")

plt.ylim(price.min() - 20, price.max() + 20)
plt.xlabel("House area (sq.ft)")
plt.ylabel("House price (lakhs)")
plt.title("Polynomial Regression: Degrees 1, 2 and 10")
plt.legend()
plt.grid(True)
plt.show()
```

Output:

```
Degree  1 -> Training MSE = 32.90
Degree  2 -> Training MSE = 22.31
Degree 10 -> Training MSE = 16.53
```



Result:

The program was executed successfully and the output was obtained as shown above.

Question 2

Question:

A college develops a machine learning model to predict students' examination marks based on study hours. The model uses a simple linear relationship, but the actual marks follow a nonlinear pattern. Write a Python program to generate nonlinear student-performance data, train a linear regression model, calculate its training and testing mean squared errors, and to compare the actual marks with the predicted values.

Aim:

To generate nonlinear student-performance data, to train a linear regression model, to calculate its training and testing mean squared errors and to compare the actual marks with the predicted values (underfitting / high bias).

Procedure:

1. Import NumPy and Matplotlib and set a random seed.
2. Generate 60 samples of study hours and non-linear (S-shaped) marks with random noise.
3. Split the data randomly into 70% training and 30% testing sets.
4. Train a linear regression model (degree-1 polynomial) on the training data using np.polyfit().
5. Predict the marks for the training and testing sets and calculate the training and testing MSE.
6. Plot the data with the fitted line and plot actual vs predicted marks for the test set; interpret the high error as underfitting.

Program:

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)

# Step 1: Generate non-linear student-performance data (S-shaped relationship)
n = 60
hours = np.random.uniform(0, 10, n)
marks = 100 / (1 + np.exp(-(hours - 5))) + np.random.normal(0, 4, n)

# Step 2: Split into training (70%) and testing (30%) sets
idx = np.random.permutation(n)
train_idx, test_idx = idx[:42], idx[42:]
x_train, y_train = hours[train_idx], marks[train_idx]
x_test, y_test = hours[test_idx], marks[test_idx]

# Step 3: Train a linear regression model  (marks = m * hours + c)
coef = np.polyfit(x_train, y_train, 1)
print("Slope =", round(coef[0], 3), " Intercept =", round(coef[1], 3))

# Step 4: Predictions and mean squared errors
pred_train = np.polyval(coef, x_train)
pred_test = np.polyval(coef, x_test)
train_mse = np.mean((y_train - pred_train) ** 2)
test_mse = np.mean((y_test - pred_test) ** 2)
print("Training MSE =", round(train_mse, 2))
```

```
print("Testing MSE  =", round(test_mse, 2))
print("Both errors are high -> the linear model underfits the non-linear data.")

# Step 5: Compare actual and predicted marks (first 5 test samples)
print("\nStudy hours   Actual marks   Predicted marks")
for h, a, p in list(zip(x_test, y_test, pred_test))[:5]:
    print(f"{h:11.2f}   {a:12.2f}   {p:15.2f}")

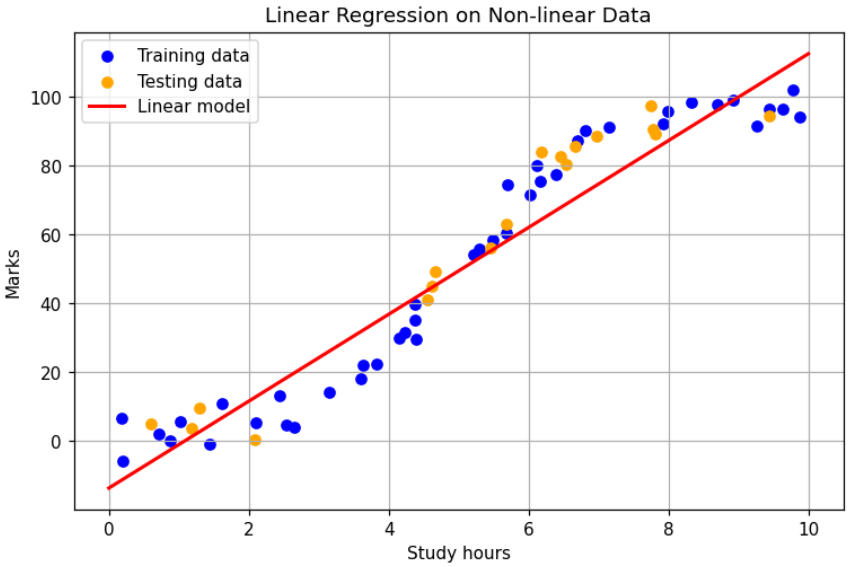
# Plot 1: data and fitted line
plt.figure(figsize=(8, 5))
plt.scatter(x_train, y_train, color="blue", label="Training data")
plt.scatter(x_test, y_test, color="orange", label="Testing data")
x_line = np.linspace(0, 10, 100)
plt.plot(x_line, np.polyval(coef, x_line), color="red", linewidth=2, label="Linear model")
plt.xlabel("Study hours")
plt.ylabel("Marks")
plt.title("Linear Regression on Non-linear Data")
plt.legend()
plt.grid(True)
plt.show()

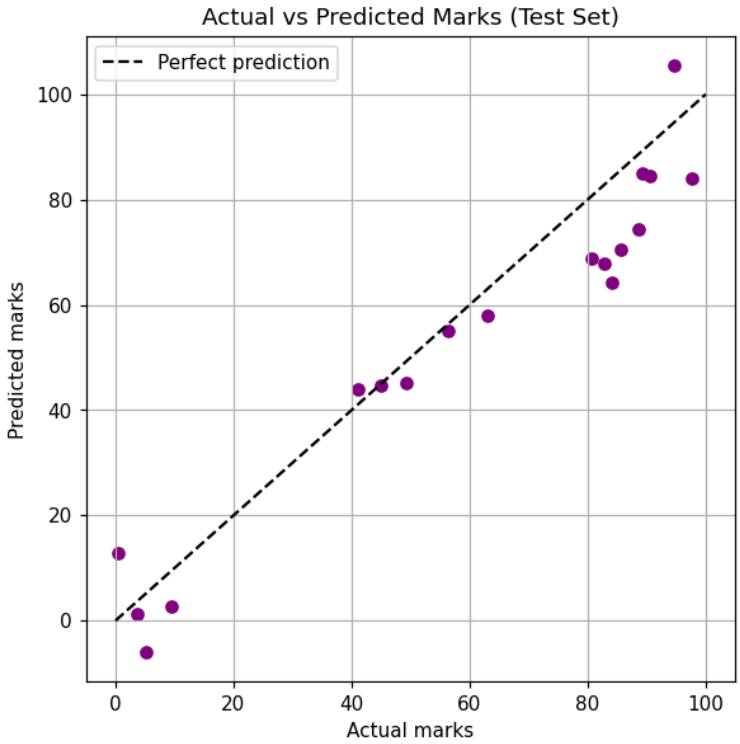
# Plot 2: actual vs predicted marks (test set)
plt.figure(figsize=(6, 6))
plt.scatter(y_test, pred_test, color="purple")
plt.plot([0, 100], [0, 100], "k--", label="Perfect prediction")
plt.xlabel("Actual marks")
plt.ylabel("Predicted marks")
plt.title("Actual vs Predicted Marks (Test Set)")
plt.legend()
plt.grid(True)
plt.show()
```

Output:

```
Slope = 12.616  Intercept = -13.603
Training MSE = 105.82
Testing MSE  = 106.48
Both errors are high -> the linear model underfits the non-linear data.

Study hours   Actual marks   Predicted marks
       4.56          41.07             43.94
       0.60           5.12             -6.01
       7.81          89.36             84.87
       6.46          82.85             67.88
       4.61          45.04             44.62
```





Result:

The program was executed successfully and the output was obtained as shown above.

Question 3

Question:

A deep learning research team uses a machine learning model to classify images based on extracted numerical features. A highly complex polynomial model performs well on training data but poorly on unseen testing data. Write a Python program to generate a sample nonlinear dataset, train polynomial regression models with degrees 2 and 15, calculate their training and testing errors, and to compare their fitted curves.

Aim:

To generate a nonlinear dataset, to train polynomial regression models of degrees 2 and 15, to calculate their training and testing errors and to compare their fitted curves (overfitting).

Procedure:

1. Import NumPy and Matplotlib and set a random seed.
2. Generate 30 samples of a non-linear (sine-shaped) dataset with random noise.
3. Split the data randomly into 60% training (18 samples) and 40% testing (12 samples) sets.
4. Train polynomial regression models of degrees 2 and 15 on the training data.
5. Calculate the training MSE and testing MSE of each model and display them in a table.
6. Plot the fitted curves of both models with the training and testing points and interpret the overfitting of the degree-15 model.

Program:

```
import warnings
import numpy as np
import matplotlib.pyplot as plt

warnings.simplefilter("ignore")
np.random.seed(5)

# Step 1: Generate a non-linear dataset  y = sin(3x) + noise
n = 30
x = np.random.uniform(-1, 1, n)
y = np.sin(3 * x) + np.random.normal(0, 0.3, n)

# Step 2: Train / test split (60% / 40%)
idx = np.random.permutation(n)
train_idx, test_idx = idx[:18], idx[18:]
x_train, y_train = x[train_idx], y[train_idx]
x_test, y_test = x[test_idx], y[test_idx]

# Step 3: Train models of degree 2 and 15 and calculate errors
degrees = [2, 15]
colors = ["green", "red"]
x_line = np.linspace(-1, 1, 400)

plt.figure(figsize=(9, 6))
plt.scatter(x_train, y_train, color="blue", label="Training data")
plt.scatter(x_test, y_test, color="orange", marker="s", label="Testing data")

print("Degree   Training MSE   Testing MSE")
for d, c in zip(degrees, colors):
    coef = np.polyfit(x_train, y_train, d)
```

```
    train_mse = np.mean((y_train - np.polyval(coef, x_train)) ** 2)
    test_mse = np.mean((y_test - np.polyval(coef, x_test)) ** 2)
    print(f"{d:6d}   {train_mse:12.4f}   {test_mse:11.4f}")
    plt.plot(x_line, np.polyval(coef, x_line), color=c, linewidth=2, label=f"Degree {d}")

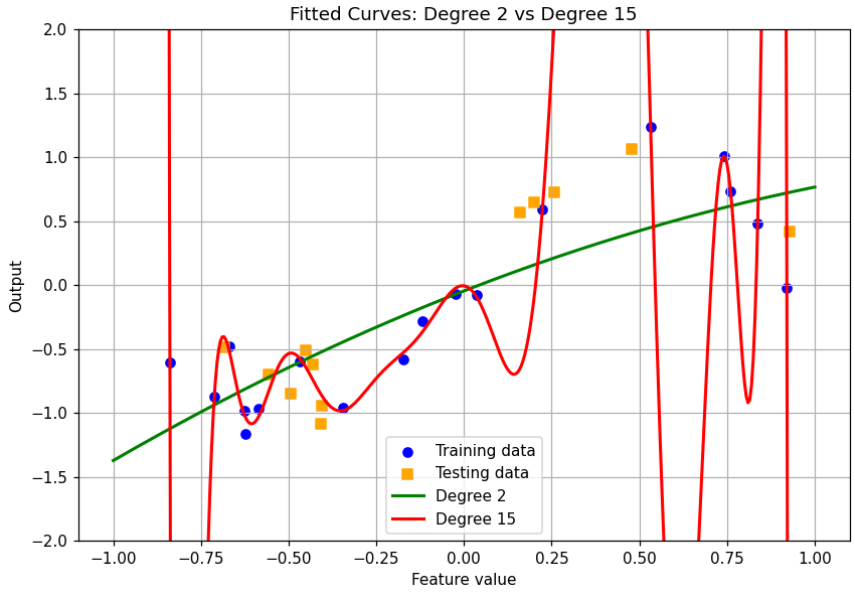
if test_mse > 2 * train_mse:
    print("\nThe degree-15 model has a very low training error but a much higher testing
error -> overfitting.")

# Step 4: Compare the fitted curves
plt.ylim(-2, 2)
plt.xlabel("Feature value")
plt.ylabel("Output")
plt.title("Fitted Curves: Degree 2 vs Degree 15")
plt.legend()
plt.grid(True)
plt.show()
```

Output:

```
Degree   Training MSE   Testing MSE
     2         0.1432        0.1622
    15         0.0021       13.7596

The degree-15 model has a very low training error but a much higher testing error ->
overfitting.
```



Result:

The program was executed successfully and the output was obtained as shown above.

Question 4

Question:

A weather forecasting team develops prediction models with different polynomial degrees to estimate temperature based on time-related features. The team wants to identify a suitable model that balances simplicity and predictive accuracy. Write a Python program to generate sample temperature data, train polynomial regression models with degrees 1, 3, and 12, calculate the training and testing mean squared errors, and to compare model performance.

Aim:

To generate sample temperature data, to train polynomial regression models of degrees 1, 3 and 12, to calculate their training and testing mean squared errors and to compare model performance in order to select a suitable model.

Procedure:

1. Import NumPy and Matplotlib and set a random seed.
2. Generate 40 samples of daytime temperature against hour of the day (6 to 18) with random noise.
3. Split the data randomly into 70% training and 30% testing sets.
4. Train polynomial regression models of degrees 1, 3 and 12 on the training data.
5. Calculate the training and testing MSE for each model and display them in a table.
6. Select the model with the lowest testing MSE as the suitable model.
7. Plot the fitted curves and a grouped bar chart of training and testing MSE.

Program:

```
import warnings
import numpy as np
import matplotlib.pyplot as plt

warnings.simplefilter("ignore")
np.random.seed(5)

# Step 1: Generate sample temperature data (hour of day 6 to 18, temperature in deg C)
n = 40
t = np.random.uniform(6, 18, n)
temp = 22 + 10 * np.sin(np.pi * (t - 6) / 12) + np.random.normal(0, 1, n)
x = (t - 12) / 6                                  # scale to [-1, 1]

# Step 2: Train / test split (70% / 30%)
idx = np.random.permutation(n)
train_idx, test_idx = idx[:28], idx[28:]
x_train, y_train = x[train_idx], temp[train_idx]
x_test, y_test = x[test_idx], temp[test_idx]

# Step 3: Train models and calculate errors
degrees = [1, 3, 12]
colors = ["red", "green", "purple"]
train_errors, test_errors = [], []
x_line = np.linspace(-1, 1, 400)

plt.figure(figsize=(9, 6))
plt.scatter(t[train_idx], y_train, color="blue", label="Training data")
```

```
plt.scatter(t[test_idx], y_test, color="orange", marker="s", label="Testing data")

print("Degree   Training MSE   Testing MSE")
for d, c in zip(degrees, colors):
    coef = np.polyfit(x_train, y_train, d)
    tr = np.mean((y_train - np.polyval(coef, x_train)) ** 2)
    te = np.mean((y_test - np.polyval(coef, x_test)) ** 2)
    train_errors.append(tr)
    test_errors.append(te)
    print(f"{d:6d}   {tr:12.4f}   {te:11.4f}")
    plt.plot(x_line * 6 + 12, np.polyval(coef, x_line), color=c, linewidth=2, label=f"Degree
{d}")

best = degrees[int(np.argmin(test_errors))]
print("\nSuitable model (lowest testing MSE): Degree", best)

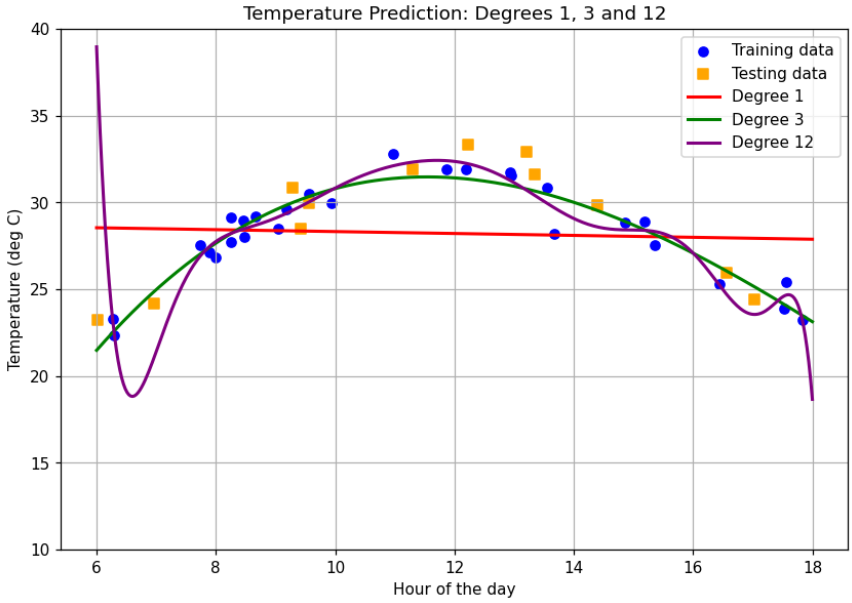
plt.ylim(10, 40)
plt.xlabel("Hour of the day")
plt.ylabel("Temperature (deg C)")
plt.title("Temperature Prediction: Degrees 1, 3 and 12")
plt.legend()
plt.grid(True)
plt.show()

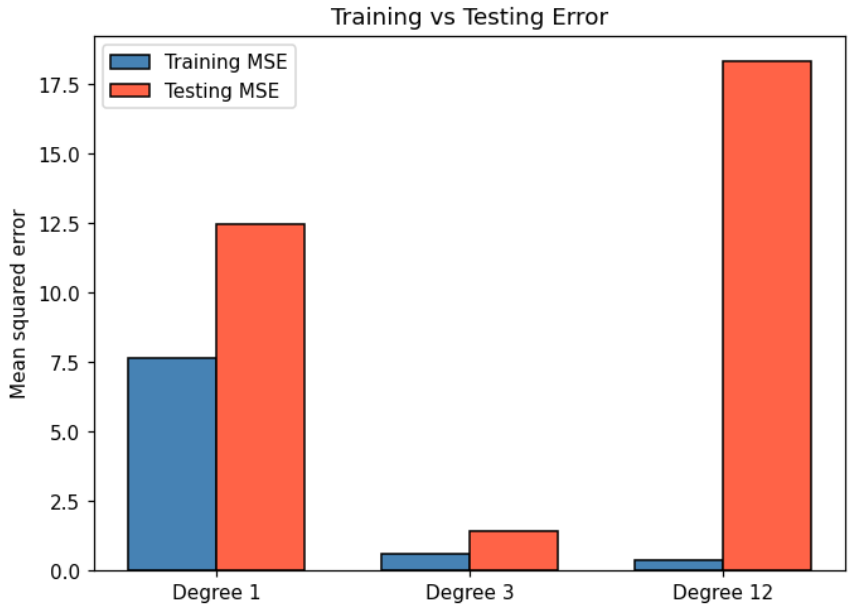
# Step 4: Compare training and testing MSE
pos = np.arange(len(degrees))
width = 0.35
plt.figure(figsize=(7, 5))
plt.bar(pos - width / 2, train_errors, width, label="Training MSE", color="steelblue",
edgecolor="black")
plt.bar(pos + width / 2, test_errors, width, label="Testing MSE", color="tomato",
edgecolor="black")
plt.xticks(pos, [f"Degree {d}" for d in degrees])
plt.ylabel("Mean squared error")
plt.title("Training vs Testing Error")
plt.legend()
plt.show()
```

Output:

```
Degree   Training MSE   Testing MSE
     1         7.6614       12.4703
     3         0.6049        1.4414
    12         0.3835       18.3355

Suitable model (lowest testing MSE): Degree 3
```





Result:

The program was executed successfully and the output was obtained as shown above.

Question 5

Question:

An online learning platform predicts student assessment scores using a machine learning model. The development team wants to understand whether the model suffers from high bias or high variance by examining its performance as the amount of training data increases. Write a Python program to generate a sample regression dataset, calculate learning curves for a linear regression model and a high-degree polynomial regression model, and to plot training and validation errors against the number of training samples.

Aim:

To generate a regression dataset, to calculate learning curves for a linear regression model and a high-degree polynomial regression model and to plot training and validation errors against the number of training samples in order to diagnose high bias or high variance.

Procedure:

1. Import NumPy and Matplotlib and set a random seed.
2. Generate a training pool of 100 samples and a separate validation set of 100 samples of a non-linear score dataset with noise.
3. Define a function that, for each training-set size m, trains a polynomial model of a given degree on the first m samples and calculates the training MSE (on those m samples) and the validation MSE (on the validation set).
4. Compute the learning curves for a linear model (degree 1) and a high-degree polynomial model (degree 10).
5. Plot training and validation errors against the number of training samples for both models.
6. Interpret the curves: high, converging errors indicate high bias; a large gap between training and validation errors indicates high variance.

Program:

```
import warnings
import numpy as np
import matplotlib.pyplot as plt

warnings.simplefilter("ignore")
np.random.seed(7)

# Step 1: Generate the regression dataset (study hours -> assessment score)
def make_data(n):
    x = np.random.uniform(0, 10, n)
    y = 40 + 3 * x + 10 * np.sin(x) + np.random.normal(0, 3, n)
    return (x - 5) / 5, y                        # x scaled to [-1, 1]

x_train, y_train = make_data(100)                # training pool
x_val, y_val = make_data(100)                    # fixed validation set

# Step 2: Learning-curve function
def learning_curve(degree, sizes):
    train_err, val_err = [], []
    for m in sizes:
        coef = np.polyfit(x_train[:m], y_train[:m], degree)
        train_err.append(np.mean((y_train[:m] - np.polyval(coef, x_train[:m])) ** 2))
```

```
        val_err.append(np.mean((y_val - np.polyval(coef, x_val)) ** 2))
    return train_err, val_err

sizes = list(range(12, 101, 4))

# Step 3: Learning curves for a linear model and a high-degree polynomial model
lin_train, lin_val = learning_curve(1, sizes)
poly_train, poly_val = learning_curve(10, sizes)

print("Linear model (degree 1) with all 100 samples:")
print("   Training MSE   =", round(lin_train[-1], 2))
print("   Validation MSE =", round(lin_val[-1], 2))
print("Polynomial model (degree 10) with only 12 samples:")
print("   Training MSE   =", round(poly_train[0], 2))
print("   Validation MSE =", round(poly_val[0], 2))
print("Polynomial model (degree 10) with all 100 samples:")
print("   Training MSE   =", round(poly_train[-1], 2))
print("   Validation MSE =", round(poly_val[-1], 2))

# Step 4: Plot the learning curves
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].plot(sizes, lin_train, "b-o", markersize=3, label="Training error")
axes[0].plot(sizes, lin_val, "r-o", markersize=3, label="Validation error")
axes[0].set_title("Linear Model (Degree 1): High Bias")

axes[1].plot(sizes, poly_train, "b-o", markersize=3, label="Training error")
axes[1].plot(sizes, poly_val, "r-o", markersize=3, label="Validation error")
axes[1].set_title("Polynomial Model (Degree 10): High Variance")

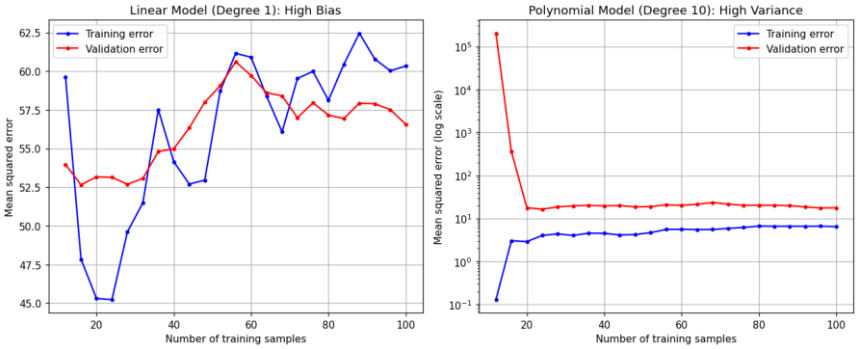
for ax in axes:
    ax.set_xlabel("Number of training samples")
    ax.set_ylabel("Mean squared error")
    ax.legend()
    ax.grid(True)

axes[1].set_yscale("log")        # errors differ by orders of magnitude -> log scale
axes[1].set_ylabel("Mean squared error (log scale)")

plt.tight_layout()
plt.show()
```

Output:

```
Linear model (degree 1) with all 100 samples:
   Training MSE   = 60.34
   Validation MSE = 56.57
Polynomial model (degree 10) with only 12 samples:
   Training MSE   = 0.13
   Validation MSE = 202127.88
Polynomial model (degree 10) with all 100 samples:
   Training MSE   = 6.48
   Validation MSE = 17.74
```



Result:

The program was executed successfully and the output was obtained as shown above.